

SHARECHATINSIGHTS: An Automated Pipeline for Analyzing Large-Scale Human-LLM Conversations

Yueru Yan* Yilin Gong* Siqi Wu Thai Le

Luddy School of Informatics, Computing, and Engineering

Indiana University Bloomington

Bloomington, IN, USA

{yueryan, yg24, swu2, tle}@iu.edu

Abstract

Large corpora of conversations between humans and large language models (LLMs) provide a valuable record of how AI systems are used in the wild, but extracting empirical insights from them requires substantial engineering to build custom pipelines that parse multi-platform chat logs and process platform-native artifacts. We present SHARECHATINSIGHTS, an interactive system that enables natural-language querying and analysis of ShareChat, a large-scale corpus of human-LLM conversations spanning five commercial assistants. An AI agent refines each request through four stages: identify, scope, model, and analyze, turning an open-ended question into a fully specified study that the system executes and exports as a runnable script. Users can configure filters to search, inspect, or export individual conversations on a separate search interface. Users can also opt to publish completed analyses to a shared community gallery. We evaluate the system on 46 requests drawn from prior literature, validate the results against independent LLM and human judgments, and present two empirical case studies. By integrating natural-language querying with reusable analytical infrastructure, SHARECHATINSIGHTS lowers the barrier to large-scale empirical research on human-LLM conversations for researchers without extensive technical skills.¹

1 Introduction

People increasingly interact with commercial conversational AI agents such as ChatGPT, Claude and Gemini. Large collections of these real-world human-LLM conversations have become a vital resource for studying how people use AI systems (Zhao et al., 2024; Zheng et al., 2024; Yan et al., 2025). However, translating these massive corpora into empirical insights remains a challenging task. To investigate nuanced phenomena, such

Attribute	ShareChat Corpus Statistics
Total Conversations	142,808
Total Turns	660,293
Platforms Covered	ChatGPT, Claude, Gemini, Perplexity, Grok
Total Languages	95
Native Artifacts	Citations, Reasoning Traces, Code Blocks

Table 1: Summary statistics of the ShareChat dataset (Yan et al., 2025). Its comprehensive, multi-platform nature drives the need for accessible analysis infrastructure.

as how users express frustration or attempt to bypass safety rules, researchers often need to build custom infrastructure for indexing, filtering, and analyzing complex conversational data. This engineering barrier could hinder researchers in computational social science, human-computer interaction, and linguistics from pursuing important research questions when they lack data-engineering expertise or computational resources.

Existing tools only partially address this barrier. Purpose-built explorers such as WildVis (Deng et al., 2024) and JailbreakHunter (Jin et al., 2025) support keyword and embedding-based search over large chat logs, but they are primarily designed for data browsing rather than analysis. Conversely, general data-analysis agents (Dibia, 2023; Wang et al., 2025; Ma et al., 2023; Hong et al., 2025) can generate analysis code, but they are corpus-agnostic and require users to manually specify data schema, sampling rules, and domain framing.

The need for an integrated, corpus-aware solution became clear following our release of the ShareChat dataset (Yan et al., 2025). As detailed in Table 1, ShareChat spans multiple commercial platforms and preserves rich native artifacts, including citations, thinking traces, and code blocks, providing opportunities for comparative and cross-platform research. Since its release, ShareChat

*Equal contribution.

¹Demo website: <https://sharechat.luddy.iu.edu/>

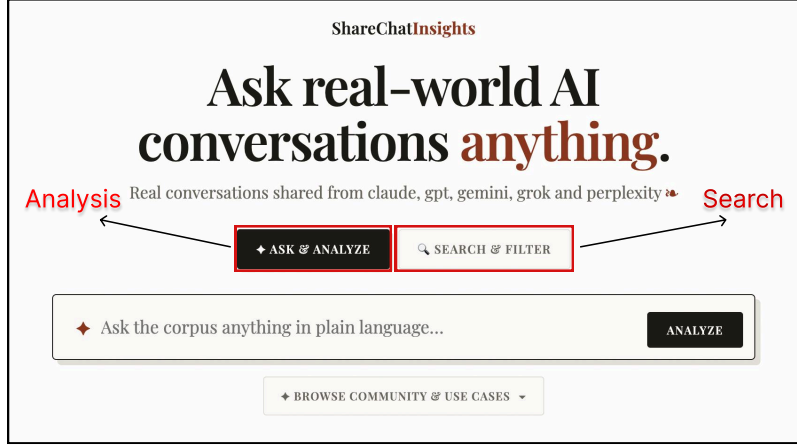


Figure 1: The landing page of SHARECHATINSIGHTS . The “Ask & Analyze” and “Search & Filter” entry points route users to the automated analysis pipeline and the search-and-browse interface, respectively.

has attracted interest from researchers across disciplines. The literature-grounded evaluation requests in Appendix B illustrate the breadth of analyses such data can support. Yet, the complexity of parsing ShareChat’s rich artifacts creates a technical barrier for researchers outside of the highly technical Natural Language Processing (NLP) field.

To make this unique resource more accessible, we develop SHARECHATINSIGHTS, an interactive system (Figure 1) that enables users to conduct multi-platform analyses through natural-language querying. The system consists of two main components: (1) an agent-supported analysis pipeline that guides users through four stages—identify, scope, model, and analyze—to refine and operationalize their research questions without writing code, and (2) a search-and-browse interface for the qualitative inspection of individual conversations. Users can also publish completed analyses to a community gallery (Appendix A), which creates a valuable shared library of reusable studies.

We evaluate the system in two complementary ways. First, we evaluate the pipeline on 46 natural-language requests by testing script execution and reproducibility, comparing descriptive and statistical outputs against fixed corpus references, and validating pipeline-generated labels against an independent LLM and human annotations. Second, we present two empirical case studies, each initiated from a natural-language request. The first reuses an established feedback codebook (Don-Yehiya et al., 2024) and reveals a divergent, corpus-specific pattern, illustrating how SHARECHATINSIGHTS can support the re-examination of prior coding schemes on new conversational data. The second identi-

fies recurring families of adversarial jailbreak tactics that align with strategies documented in prior work (Shen et al., 2024; Wei et al., 2023).

By integrating an agent-guided computational analysis pipeline with interactive data browsing, SHARECHATINSIGHTS supports a broader range of research tasks for users across disciplines. The system is available as a free hosted service for research use, and all use of the ShareChat corpus adheres to the license terms of its original release (Yan et al., 2025).

2 Related Work

Systems for exploring conversational corpora. WildVis (Deng et al., 2024) supports keyword-filtered and embedding-based search with visualization over millions of chat logs, enabling exploratory browsing of datasets such as WildChat and LMSYS-Chat-1M. ConvoKit (Chang et al., 2020) provides a programmatic toolkit of conversational analysis primitives over a unified corpus format. It was developed primarily for human-human dialogue and does not provide an interactive interface. WIMBD (Elazar et al., 2024) offers count- and search-based analyses that surface corpus statistics, data-quality issues, and benchmark contamination at scale, but targets pretraining corpora rather than conversational data. Clio (Tamkin et al., 2024) performs automated, privacy-preserving clustering of real-world usage data, but operates as an internal platform over proprietary data. Among publicly available interactive systems, WildVis is the closest to our setting of large-scale human-LLM conversational analysis, but it focuses on search, browsing, and visualization rather than gen-

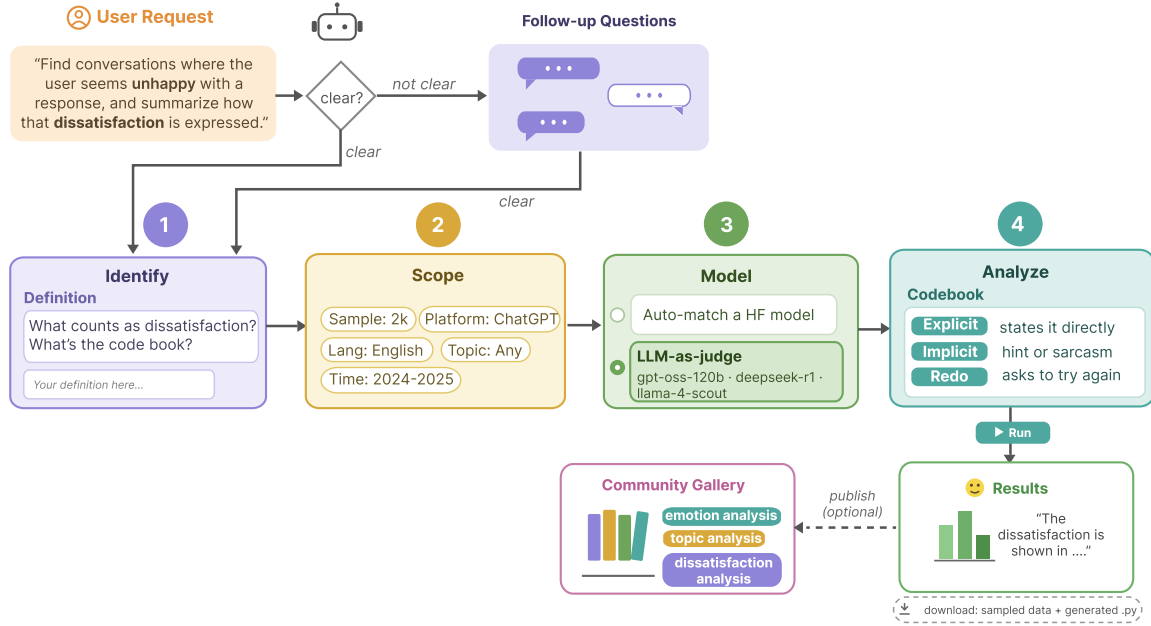


Figure 2: Overview of the automated analysis pipeline. A natural-language request first undergoes a clarity check with follow-up questions as needed, and is then refined through the identify, scope, model, and analyze stages into an executable analysis. Results can be downloaded and optionally published to the shared community gallery.

erating executable computational analyses.

LLM agents for data analysis. A parallel line of work applies LLMs to automate data analysis and visualization from natural language requests. LIDA (Dibia, 2023) decomposes visualization generation into a multi-stage LLM pipeline that summarizes a dataset, enumerates visualization goals, and generates and refines the corresponding chart code. Data Formulator (Wang et al., 2025) allows users to pose natural-language questions over connected data sources and edit, branch, and share the resulting charts on an interactive canvas. Insight-Pilot (Ma et al., 2023) automates exploration by having an LLM select analysis intents and issue structured queries to an insight engine, producing a coherent sequence of insights. General code-interpreter agents translate free-form requests into executable analysis code (Hong et al., 2025; Qiao et al., 2023). These systems are corpus-agnostic and operate at the level of individual requests. SHARECHATINSIGHTS builds upon both lines of work by combining interactive corpus browsing with agent-guided computational analysis.

3 System Design

Our system consists of two main components, both shown in Figure 1: an automated, agent-guided analysis pipeline and a search-and-browse interface.

Users can first browse the corpus to build intuitions, and then launch a computational analysis once they have identified a question of interest.

3.1 Ask & Analyze: Agent-Guided Pipeline

The automated analysis component enables users to study the large-scale human–LLM conversations in ShareChat through natural-language requests, without writing code. Users first describes the analysis they want to conduct, and the system guides them through a four-stage pipeline that we call *identify*, *scope*, *model*, and *analyze*. Each stage turns an open-ended query into an explicit analysis specification, ensuring that key parameters are recorded and the analysis can be reproduced. The full pipeline is illustrated in Figure 2.

The process begins when a user submits a natural-language request. An LLM agent first determines whether the request is sufficiently specific to act on. If not, it identified the core task from the request and asks follow-up questions to clarify the analysis plan. Once the plan is sufficiently specified, the pipeline proceeds through the four stages described below.

- **Identify.** The agent confirms what the analysis will measure. For example, given the request “Find conversations where the user seems unhappy with a response, and summa-

size how that dissatisfaction is expressed,” the agent asks the user to specify what counts as dissatisfaction and how it should be operationalized. At this stage, the user could upload a custom codebook to the agent. This step resolves ambiguity before computation begins, ensuring that the resulting analysis reflects the user’s intended construct.

- **Scope.** The user selects the subset of the corpus to analyze. They can restrict the analysis by sample size, platform, language, topic, and time frame. This allows users to first conduct a pilot analysis on a small sample, and then scale the same analysis to the full corpus.
- **Model.** The system determines how the analysis will be conducted. When a suitable pre-trained model is readily available on Hugging Face (HF), SHARECHATINSIGHTS automatically selects the best-matching model. For instance, if the user requests emotion analysis, the system searches for emotion-classification models on Hugging Face. If no suitable model is available, the user can instead opt for an LLM-as-a-judge approach (Zheng et al., 2023), in which a selected LLM reads each conversation and assigns labels according to the user’s definition. For this option, we support three models: gpt-oss-120b (Agarwal et al., 2025), deepseek-r1, and llama-4-scout.
- **Analysis.** The system presents the final analysis plan for the user to review and confirm before sending it to the analysis engine. When the plan involves an LLM as a judge, the system also displays the prompt that will be sent to the judge model, allowing the user to review it before execution. At this stage, the user can also download the sampled conversations and generated Python code, if they prefer to run the analysis in their local environment.

After the entire analysis plan is submitted, a background worker executes the analysis. Once the run is complete, the system presents the results as charts accompanied by text descriptions. Users can also download the results locally. To help new users get started, we provide three sample analyses as working examples. In addition, users may contribute their completed analyses to the community gallery, so that other users can inspect, reuse, build

on prior analyses. Over time, this gallery can grow into a shared resource that broadens the range of analyses supported by the system.

3.2 Search & Filter

The search-and-browse interface allows users to locate and inspect individual conversations. Users can search by keyword and restrict the matches to the user’s messages, the LLM’s messages, the entire conversation, or the model’s thinking trace. Results can be further narrowed using filters for platform, language, topic, interface type (e.g., thinking bar, links, sources, or code), and time frame. The interface then returns a list of conversations that satisfy these criteria. Selecting a conversation opens its detail page, which displays every turn exchanged between the user and the LLM, along with associated metadata such as platform, language, and timestamp. Each conversation also links to its original publicly shared source. These features help users understand the corpus before deciding what analyses to conduct.

3.3 System Architecture

The system architecture is shown in Figure 3. A session begins when a user submits a natural-language request through the frontend, a browser-based interface built with HTML, CSS, and JavaScript. The request is routed to a Flask backend, which sends it to a planner LLM (gpt-oss-120b), served by an external LLM Inference Service through an OpenAI-compatible endpoint. The planner distinguishes between two types of requests and routes them to different execution paths. The first consists of deterministic requests, whose answer is exactly computable from the corpus, such as counting records, computing summary statistics, filtering entries, or aggregating values. For these requests, the planner identifies the required database operations, which are then executed using SQL against the corpus database. The second type consists of requests that require more general or multi-step analysis. For these requests, the planner produces an analysis plan, which the backend validates and executes using the supported analysis operations. The compiled queries run against a corpus database implemented in SQLite with full-text search (FTS5), enabling efficient keyword filtering over the full dataset. Long-running analyses are handled asynchronously by a decoupled background worker that executes jobs from a queue with bounded concurrency, preventing analysis workloads from block-

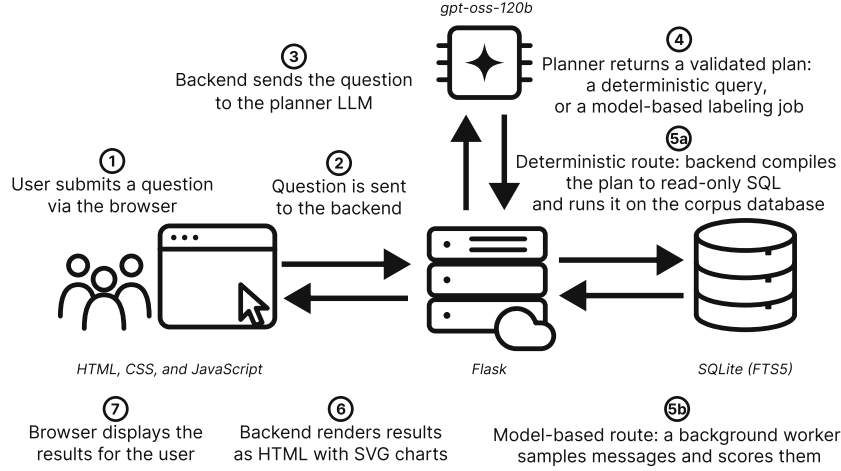


Figure 3: Overview of the data flow from request submission to result rendering. The planner returns a validated plan that the backend routes to one of two paths: deterministic requests compile to read-only SQL over the corpus, while interpretive requests are dispatched to a background worker that samples messages, sends the selected text and confirmed codebook to the chosen model, and aggregates the returned labels.

ing interactive web requests. Once a job is complete, the backend aggregates the results and renders them as HTML with SVG charts, which are returned to the browser and displayed to the user.

4 Evaluation

We evaluate SHARECHATINSIGHTS along two complementary dimensions: the technical reliability of its automated analysis pipeline and its ability to support substantive empirical research. Section 4.1 examines pipeline reliability and correctness by testing whether generated scripts execute successfully, whether descriptive and statistical outputs match independently computed corpus references, and whether model-generated labels agree with an independent LLM and human annotators. We evaluate these properties using a curated suite of 46 requests grounded in published analytic frameworks and codebooks. Section 4.2 then examines whether analyses produced by the system can support substantive empirical inquiry. We present two case studies, each initiated from a single natural-language request. The first applies an established feedback codebook and reveals a corpus-specific pattern that diverges from prior findings, while the second identifies recurring families of adversarial jailbreak tactics.

4.1 Pipeline Evaluation

To assess the pipeline’s coverage and correctness, we evaluate SHARECHATINSIGHTS on 46 natural-language requests (Appendix B). The evaluation

suite includes 35 descriptive and statistical tasks drawn from Amar et al. (2005) and InfiAgent-DABench (Hu et al., 2024), along with 11 codebook-based labeling tasks adapted from prior work. Because codebook application is widely used in both content analysis and computer-mediated discourse analysis (Krippendorff, 1989; Herring, 2018), these labeling tasks cover a broad range of phenomena, including linguistic interaction, user behavior and intent, and AI safety (Solorio et al., 2014; Danescu-Niculescu-Mizil et al., 2013; Nass and Moon, 2000; Stolcke et al., 2000; Ho et al., 2018; Socher et al., 2013; Zhao et al., 2024; Jiang et al., 2024; Tamkin et al., 2024). For the labeling tasks, SHARECHATINSIGHTS uses gpt-oss-120b with temperature ($T=0$).

Evaluation protocol. We report two measures. *Executability* is the percentage of generated scripts that run successfully and return a structured result. *Reference agreement* measures whether the output of each script matches an independently computed reference for the original request.

We categorize the 46 requests into three tiers, as shown in Table 2. Tier A consists of descriptive requests that retrieve, rank, aggregate, or characterize distributions of conversation metadata, while Tier B consists of statistical requests such as outlier detection, correlation, regression, and group comparisons. Because Tier A and B requests have fixed answers that can be verified directly from the corpus, we evaluate reference agreement by comparing each returned result against an independently

Task set	Req.	Decl.	Scored	Exec.	Ref. agr.
A (Descriptive)	24	3	21	100.0%	100.0%
B (Statistical)	11	1	10	100.0%	100.0%
C (Labeling)	11	0	11	100.0%	71.6%
Total	46	4	42	100.0%	—

Table 2: Pipeline evaluation on the curated suite. *Decl.* counts requests the validation gate declined with a stated reason, plus one request for which the planner produced no valid plan. Tier C agreement is item-level over 550 sampled items (50 per task).

computed corpus reference. Requests that the system declines with a stated reason are reported separately, as is the single request for which the planner fails to produce a valid plan. Both failure cases are excluded from the reference-agreement denominator. Tier C consists of requests that require interpretive labels rather than quantities with fixed answers. For each of the 11 Tier C tasks, we therefore randomly sample 50 items and ask Claude Opus 4.7 to independently apply the same codebook without access to the system’s predictions. For Tier C, reference agreement is the proportion of the 550 sampled items for which the system and the independent LLM judge assign the same label.

As shown in Table 2, across the 35 Tier A and B requests, 31 were successfully answered, three were declined, and one failed to produce a valid plan. All 31 substantive results matched their fixed reference answers, yielding 100% reference agreement. The validation check declined plans when the proposed computations did not adequately address the original request. For example, one request asked for the standard deviation and histogram, but the proposed plan would have returned a turn count. All four failed requests were successfully resolved upon rerunning the pipeline.

For Tier C, the system agreed with the independent LLM judge on 71.6% of the 550 sampled items (95% CI [68%, 75%]). Agreement varied substantially across tasks. It was highest for codebooks based on observable properties, reaching 96.0% for code-switching, 90.0% for residual PII, and 88.0% for self-disclosure. It was lowest for tasks requiring inferences about user intent, reaching 56.0% for primary use case; politeness and refusal each reached 58.0%. In these cases, the system and LLM judge applied the same codebook to the same items but produced different labels. We therefore cannot attribute the disagreement to the system alone: it may also reflect underspecified

category boundaries, ambiguity in the underlying construct, or both.

Because the above reference labels were generated by an LLM, we also conducted a human validation. Two annotators independently coded 200 messages randomly sampled across four tasks and adjudicated their disagreements. Two are Tier C tasks (#45 and #46 in Appendix B) and two are the case studies in Section 4.2. The mean of human–human inter-rater reliability κ was 0.61. Across these 200 items, the system agreed with the adjudicated human reference on 79.0% of labels (95% CI [70%, 86%]), compared with 81.5% agreement between human annotators before adjudication.

Reproducibility of exported scripts For each analysis, SHARECHATINSIGHTS generates a downloadable script that users can run independently in their local environment. We test whether the generated script reproduces the result displayed in the demo. For Tiers A and B, the script executes the same validated plan as the demo and returns identical results. Tier C, however, relies on stochastic LLM inference, so repeated execution of the same plan on the same data sample with the same codebook may yield different item-level labels. We therefore compare the aggregate label distributions produced by the demo and the generated script, measuring their overlap as the sum of the minimum proportion for each label. Across the eight turn-level tasks, the mean overlap is 96.9%. An overlap of 100% indicates that two pipeline runs produce identical label proportions, though individual items may still receive different labels.

4.2 Case Studies

Characterizing user dissatisfaction. The request “*Find multi-turn conversations where the user seems unhappy with a response, and summarize how that dissatisfaction is expressed and where in the conversation it shows up*” examines a finding from prior research: expressions of user dissatisfaction tend to occur in the later turns of a conversation (Liu et al., 2025). At the *identify* stage, instead of allowing the agent to draft its own definition, we supplied the feedback codebook from Don-Yehiya et al. (2024). The codebook identifies user turns that rephrase or clarify a previous request, or correct a previous response, alongside an explicit positive-feedback category. We treat the four non-positive categories as implicit expressions of dissatisfaction. Reusing the established coding scheme

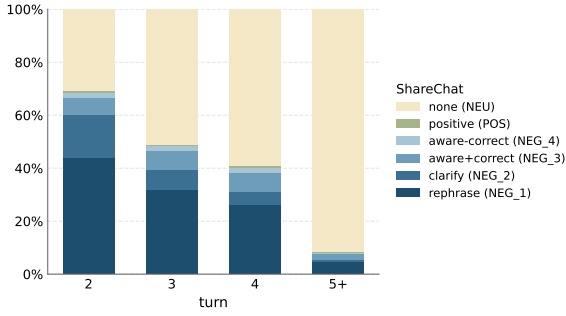


Figure 4: Turn-level distribution of user dissatisfaction. Dissatisfaction peaks at the second user turn, and declines monotonically thereafter, in contrast to the late-turn pattern reported for WildChat and LMSYS using this codebook (Liu et al., 2025).

allows us to compare patterns in ShareChat directly with findings from prior work. Because no pre-trained classifier suits this task, the pipeline uses an LLM (llama-4-scout, $T=0$) to classify individual user turns over the full English subset of 70,590 conversations. It flags 114,922 turns, or 21.3% of user messages: *repeat/rephrase* accounts for 62.8%, followed by *clarification* (16.4%), *aware with correction* (15.3%), *aware without correction* (4.5%), and *positive feedback* (0.9%).

The turn-position distribution, shown in Figure 4, differs from the pattern reported in (Liu et al., 2025). We find that dissatisfaction is most frequent at the second user turn—the earliest point at which it can occur—where it appears in 69.1% of messages, and declines monotonically to 8.4% by fifth turn or later. Because we use the same codebook and LLM judge, the observed divergence is less likely to result from differences in the coding procedure. One possible explanation is a selection effect: conversations that fail early may be disproportionately represented among those that users choose to share. The result is therefore not a refutation of prior work, but a potential corpus-level selection effect. This example illustrates how SHARECHATINSIGHTS can be used to test such hypotheses at scale.

Categorizing jailbreak tactics. The request “Find conversations that look like attempts to get around the model’s safety rules, and group them by the strategy the user is using” asks for a categorical characterization of jailbreak behavior. Prior analyses of in-the-wild chat logs have documented recurring tactic families, including role-play framing and prompt injection (Shen et al., 2024). In this

case, no predefined codebook is supplied. An LLM (gpt-oss-120b, $T=0$) first scores every user message for circumvention intent and assigns a strategy label. The agent then consolidates into broader categories (Appendix C). Over the same English subset, 2.9% of messages exceed the default threshold, and four tactics emerge: *instruction injection* (62.8%), *prompt engineering* (27.6%), *role-play* (8.3%), and *context manipulation* (1.4%). These attempts are concentrated in the first user turn.

The jailbreak tactics the system recovers, along with the canonical trigger phrases associated with each, correspond to strategies documented in prior analyses (Shen et al., 2024), in which the authors identify prompt injection and privilege escalation as major attack strategies. However, the two taxonomies are not directly comparable because Shen et al. (2024) cluster prompts into semantic communities rather than a fixed set of tactics. We thus compare the results at the level of broader strategy families. Each recovered family can be related to one of the two failure modes of safety training identified by Wei et al. (2023): competing objectives and mismatched generalization.

5 Conclusion

We introduce SHARECHATINSIGHTS, an interactive system for analyzing the multi-platform ShareChat corpus through natural-language requests. Its agent-guided pipeline transforms open-ended questions into executable, downloadable analyses, while a complementary search-and-browse interface supports the inspection of individual conversations. We evaluate the system on 46 requests from prior literature and validated its model-generated labels against an independent LLM judge and human annotations. All substantive descriptive and statistical outputs match independently computed reference labels. Two case studies further demonstrate how SHARECHATINSIGHTS can support empirical research: one reuses an established feedback codebook and reveals a corpus-specific pattern that diverges from prior findings, while the other identifies recurring families of adversarial jailbreak tactics. SHARECHATINSIGHTS lowers the barrier to large-scale research on human-LLM interaction across NLP, computational social science, and HCI. Future work will expand the set of supported judge models and improve the community gallery to facilitate the reuse and adaptation of shared analyses.

6 Limitations

ShareChat contains only conversations that users chose to share publicly and therefore does not constitute a representative sample of human–LLM interaction. The corpus may overrepresent particular topics, languages, platforms, and user populations, and both case studies use only English conversations. Although the 46 evaluation requests are grounded in prior work, they were formulated by the project team and may not reflect how researchers unfamiliar with the system would naturally phrase or scope their questions. The resulting findings should therefore be interpreted within this corpus and evaluation setting rather than generalized to human–LLM interaction more broadly.

Several analyses rely on LLM-based annotation, which can be inconsistent across runs and may even inherit biases from the underlying model. Our human validation covers four tasks and therefore does not establish accuracy across all evaluated labeling tasks. The system now routes requests between a deterministic path and a model-based path, preferring deterministic operations where the requested quantity is exactly computable. We do not yet measure how often this routing decision is correct: the evaluation suite contains no requests for which the intended behavior is to decline or to fall back to a deterministic operation, so routing accuracy on naturally phrased requests remains untested.

Ethics Considerations

Data provenance and privacy. SHARECHATINSIGHTS operates exclusively on the public release of the ShareChat corpus, which consists of conversations that users chose to share publicly and that were processed to remove personally identifiable information (PII). We use these data in accordance with the usage conditions of the original ShareChat release (Yan et al., 2025). Our system collects no new user data and adds no capability to re-identify individuals. We nonetheless recognize that publicly shared conversations can still contain sensitive content. The web interface displays aggregate analysis results, while downloaded samples and search results contain only records already included in the public, PII-processed release. Each record links to its original publicly shared source rather than exposing new material.

We also recognize that aggregating this material into a single searchable interface increases its discoverability. Full-text search, fine-grained fil-

tering, conversation-level browsing, downloadable samples, and links to original sources can make sensitive content easier to locate than when the same conversations are distributed across five separate platforms. We consider this as a trade-off for research access to a public, PII-removed corpus, while recognizing that it represents a genuine privacy cost rather than a neutral change. We will therefore restrict or remove access to any conversation upon request from the original dataset or the relevant platform.

Dual use. One case study categorizes attempts to circumvent model safety measures. Because this analysis operates over public data, reports only aggregate patterns rather than reusable attack recipes, and is intended to support defensive red-teaming and safety research, we believe its benefit to the community outweighs its risk. We ask that users of SHARECHATINSIGHTS apply the same principles.

Acknowledgments

This work used Jetstream2 at Indiana University through allocation #CIS250163 and #CIS260332 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program, which is supported by National Science Foundation grants #2138259, #2138286, #2138307, #2137603, and #2138296. The open-weight model inference used by the system and case studies was provided by the Jetstream2 LLM Inference Service at Indiana University (Hancock et al., 2021; Boerner et al., 2023). We thank UITS Research Technologies at Indiana University for providing a public IP address, networking support, and additional computing resources used to deploy and operate SHARECHATINSIGHTS.

References

- Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, et al. 2025. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*.
- Robert Amar, James Eagan, and John Stasko. 2005. Low-level components of analytic activity in information visualization. In *IEEE Symposium on Information Visualization, 2005. INFOVIS 2005.*, pages 111–117. IEEE.
- Timothy J. Boerner, Stephen Deems, Thomas R. Furlani, Shelley L. Knuth, and John Towns. 2023. [ACCESS](#):

- Advancing innovation: NSF's advanced cyberinfrastructure coordination ecosystem: Services & support. In *Practice and Experience in Advanced Research Computing*. Association for Computing Machinery.
- Jonathan P Chang, Caleb Chiam, Liye Fu, Andrew Wang, Justine Zhang, and Cristian Danescu-Niculescu-Mizil. 2020. Convokit: A toolkit for the analysis of conversations. In *Proceedings of the 21th annual meeting of the special interest group on discourse and dialogue*, pages 57–60.
- Cristian Danescu-Niculescu-Mizil, Moritz Sudhof, Dan Jurafsky, Jure Leskovec, and Christopher Potts. 2013. A computational approach to politeness with application to social factors. In *Proceedings of the 51st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 250–259, Sofia, Bulgaria. Association for Computational Linguistics.
- Yuntian Deng, Wenting Zhao, Jack Hessel, Xiang Ren, Claire Cardie, and Yejin Choi. 2024. Wildvis: Open source visualizer for million-scale chat logs in the wild. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 497–506.
- Victor Dibia. 2023. Lida: A tool for automatic generation of grammar-agnostic visualizations and infographics using large language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pages 113–126.
- Shachar Don-Yehiya, Leshem Choshen, and Omri Abend. 2024. Naturally occurring feedback is common, extractable and useful. *arXiv preprint arXiv:2407.10944*.
- Yanai Elazar, Akshita Bhagia, Ian Helgi Magnusson, Abhilasha Ravichander, Dustin Schwenk, Alane Suhr, Evan Pete Walsh, Dirk Groeneveld, Luca Soldaini, Sameer Singh, Hannaneh Hajishirzi, Noah A. Smith, and Jesse Dodge. 2024. What's in my big data? In *The Twelfth International Conference on Learning Representations*.
- David Y. Hancock, Jeremy Fischer, John Michael Lowe, Winona Snapp-Childs, Marlon Pierce, Suresh Marru, J. Eric Coulter, Matthew Vaughn, Brian Beck, Nirav Merchant, Edwin Skidmore, and Gwen Jacobs. 2021. Jetstream2: Accelerating cloud computing via jetstream. In *Practice and Experience in Advanced Research Computing*, pages 1–8. Association for Computing Machinery.
- Susan C Herring. 2018. The coevolution of computer-mediated communication and computer-mediated discourse analysis. In *Analyzing digital discourse: New insights and future directions*, pages 25–67. Springer.
- Annabell Ho, Jeff Hancock, and Adam S. Miner. 2018. Psychological, relational, and emotional effects of self-disclosure after conversations with a chatbot. *Journal of Communication*, 68(4):712–733.
- Sirui Hong, Yizhang Lin, Bang Liu, Bangbang Liu, Binhao Wu, Ceyao Zhang, Danyang Li, Jiaqi Chen, Jiayi Zhang, Jinlin Wang, et al. 2025. Data interpreter: An llm agent for data science. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 19796–19821.
- Xueyu Hu, Ziyu Zhao, Shuang Wei, Ziwei Chai, Qianli Ma, Guoyin Wang, Xuwu Wang, Jing Su, Jingjing Xu, Ming Zhu, Yao Cheng, Jianbo Yuan, Jiwei Li, Kun Kuang, Yang Yang, Hongxia Yang, and Fei Wu. 2024. InfiAgent-DABench: Evaluating agents on data analysis tasks. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 19544–19572. PMLR.
- Ken Hyland. 2005. Stance and engagement: A model of interaction in academic discourse. *Discourse Studies*, 7(2):173–192.
- Liwei Jiang, Kavel Rao, Seungju Han, Allyson Ettinger, Faeze Brahman, Sachin Kumar, Niloofar Miresghalah, Ximing Lu, Maarten Sap, Yejin Choi, et al. 2024. Wildteaming at scale: From in-the-wild jailbreaks to (adversarially) safer language models. *Advances in Neural Information Processing Systems*, 37:47094–47165.
- Zhihua Jin, Shiyi Liu, Haotian Li, Xun Zhao, and Huamin Qu. 2025. Jailbreakhunter: a visual analytics approach for jailbreak prompts discovery from large-scale human-llm conversational datasets. *IEEE Transactions on Visualization and Computer Graphics*, 31(10):7923–7937.
- Klaus Krippendorff. 1989. Content analysis. *International encyclopedia of communication*, (226):403–407.
- Yuhan Liu, Michael JQ Zhang, and Eunsol Choi. 2025. User feedback in human-llm dialogues: a lens to understand users but noisy as a learning signal. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 2666–2681.
- Pingchuan Ma, Rui Ding, Shuai Wang, Shi Han, and Dongmei Zhang. 2023. Insightpilot: An llm-empowered automated data exploration system. In *Proceedings of the 2023 conference on empirical methods in natural language processing: system demonstrations*, pages 346–352.
- Clifford Nass and Youngme Moon. 2000. Machines and mindlessness: Social responses to computers. *Journal of Social Issues*, 56(1):81–103.
- Bo Qiao, Liqun Li, Xu Zhang, Shilin He, Yu Kang, Chaoyun Zhang, Fangkai Yang, Hang Dong, Jue Zhang, Lu Wang, et al. 2023. Taskweaver: A code-first agent framework. *arXiv preprint arXiv:2311.17541*.

- Xinyue Shen, Zeyuan Chen, Michael Backes, Yun Shen, and Yang Zhang. 2024. "do anything now": Characterizing and evaluating in-the-wild jailbreak prompts on large language models. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 1671–1685.
- Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 conference on empirical methods in natural language processing*, pages 1631–1642.
- Thamar Solorio, Elizabeth Blair, Suraj Maharjan, Steven Bethard, Mona Diab, Mahmoud Ghoneim, Abdelati Hawwari, Fahad AlGhamdi, Julia Hirschberg, Alison Chang, et al. 2014. Overview for the first shared task on language identification in code-switched data. In *Proceedings of the first workshop on computational approaches to code switching*, pages 62–72.
- Andreas Stolcke, Klaus Ries, Noah Coccaro, Elizabeth Shriberg, Rebecca Bates, Daniel Jurafsky, Paul Taylor, Rachel Martin, Carol Van Ess-Dykema, and Marie Meteer. 2000. [Dialogue act modeling for automatic tagging and recognition of conversational speech](#). *Computational Linguistics*, 26(3):339–374.
- Alex Tamkin, Miles McCain, Kunal Handa, Esin Durmus, Liane Lovitt, Ankur Rathi, Saffron Huang, Alfred Mountfield, Jerry Hong, Stuart Ritchie, et al. 2024. Clio: Privacy-preserving insights into real-world ai use. *arXiv preprint arXiv:2412.13678*.
- Chenglong Wang, Bongshin Lee, Steven M Drucker, Dan Marshall, and Jianfeng Gao. 2025. Data formulator 2: Iterative creation of data visualizations, with ai transforming data along the way. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pages 1–17.
- Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. 2023. Jailbroken: How does llm safety training fail? *Advances in neural information processing systems*, 36:80079–80110.
- Yueru Yan, Tuc Nguyen, Bo Su, Melissa Lieffers, and Thai Le. 2025. Sharechat: A dataset of chatbot conversations in the wild. *arXiv preprint arXiv:2512.17843*.
- Wenting Zhao, Xiang Ren, Jack Hessel, Claire Cardie, Yejin Choi, and Yuntian Deng. 2024. [Wildchat: 1m chatGPT interaction logs in the wild](#). In *The Twelfth International Conference on Learning Representations*.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zhuohan Li, Zi Lin, Eric Xing, Joseph E. Gonzalez, Ion Stoica, and Hao Zhang. 2024. [LMSYS-chat-1m: A large-scale real-world LLM conversation dataset](#). In *The Twelfth International Conference on Learning Representations*.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623.

A Community Gallery

The ShareChatInsights community gallery serves as a collaborative repository designed to facilitate data reusability and methodological sharing. As researchers utilize the platform to investigate the corpus, they can publish their finalized execution pipelines and visual insights directly to this public interface. We anticipate that this gallery will expand continuously as the interdisciplinary user base contributes novel use cases, transforming individual queries into reusable analytical templates for the broader research community.

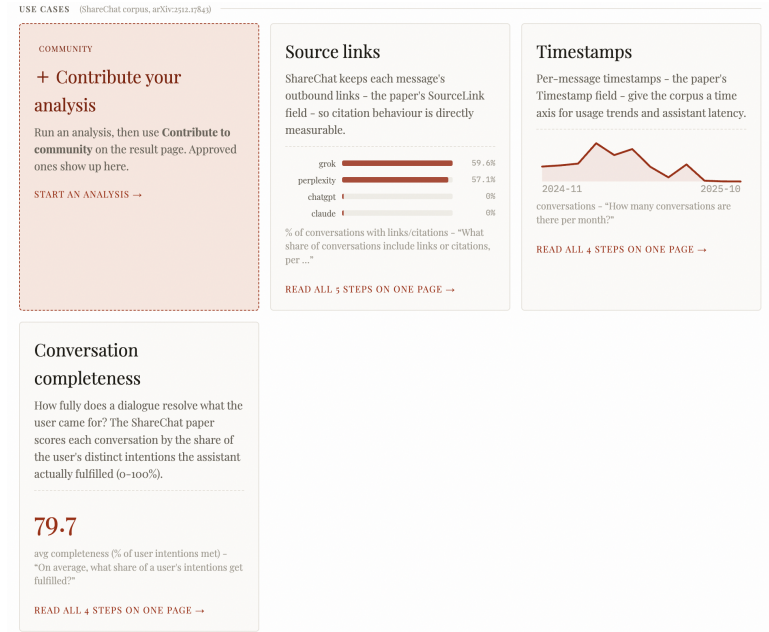


Figure 5: The ShareChatInsights Community Gallery. The interface displays published analyses as accessible cards—such as queries on source link frequency, timestamp distributions, and conversation completeness—while explicitly prompting users to contribute their own analytical pipelines.

B Evaluation Requests

Table 3 lists the 46 natural-language requests used in the pipeline evaluation (Section 4.1). Tier A (descriptive) and Tier B (aggregation and statistics) are constructed to cover the low-level analytic primitives of Amar et al. (2005) together with the data-analysis concepts of InfiAgent-DABench (Hu et al., 2024). Each Tier C (labeling) request specifies a fixed codebook and is grounded in prior work on conversational and interaction analysis, adapted to this corpus.

Table 3: The 46 natural-language requests used in the pipeline evaluation.

#	Request	Grounding
<i>Tier A + B — Descriptive and Aggregative Studies</i>		
1	Look up the conversation at a given url and return its platform, turns_count, and detected_language_final.	Amar et al. (2005)
2	For a given Gemini conversation url, return the model and published_at.	
3	For a given ChatGPT conversation url, return the model and create_time.	
4	For a given Perplexity conversation url, return its views and shares.	
5	Find the conversation with the maximum turns_count; report its platform and url.	
6	Among Perplexity conversations, find the one with the greatest shares; return its url and views.	
7	Identify the detected_language_final value with the most conversations (the modal language).	
8	Rank the five platforms by mean turns_count, descending.	
9	Return the top 20 Perplexity conversations by views, descending.	
10	Sort the topic categories by number of conversations, descending.	

Continued on next page

Table 3 — continued

#	Request	Grounding
11	Report the earliest and latest create_time among ChatGPT conversations.	Amar et al. (2005); Hu et al. (2024)
12	Report the minimum and maximum turns_count for Grok conversations.	
13	Report the range (min, max) of views across Perplexity conversations.	
14	Compute the average turns_count per platform.	
15	Compute the mean total conversation length in characters (from plain_text) per platform.	
16	Compute the fraction of Claude conversations containing at least one non-empty code artifact.	
17	Derive user-message length from plain_text and report the mean per platform.	
18	Compute, per platform, the proportion of conversations whose turns_count is below the corpus median.	
19	Characterize the distribution of turns_count for ChatGPT (histogram, mean, median, SD).	
20	Characterize the detected_language_final distribution; report the top 15 languages with percentages.	
21	Characterize the topic distribution per platform (topic-by-platform cross-tabulation).	
22	Characterize ChatGPT conversation volume by month using create_time.	
23	Characterize the distribution of the number of non-empty code artifacts per Claude conversation.	
24	Bin turns_count into ranges (1–2, 3–5, 6–10, 11+) and report the share per platform.	
25	Detect anomalous daily-volume spikes in ChatGPT create_time (days > mean + 2 SD).	
26	Flag conversations whose derived user token length exceeds the mean + 3 SD.	
27	Flag detected_language_final values appearing in only a single conversation.	
28	Fit a linear regression of monthly ChatGPT conversation count against time; report slope and R ² .	
29	Report the median and IQR of turns_count per platform.	
30	Compute, per platform, the share of English vs. non-English conversations.	
31	Test whether turns_count is associated with a non-empty thinking trace (Grok; point-biserial).	
32	Measure the correlation between views and shares (Perplexity).	
33	Test whether Claude turns_count is associated with containing a non-empty code artifact.	
34	Measure the correlation between derived user-message length and turns_count.	
35	Compare mean turns_count between English and non-English conversations.	
Tier C — Labeling (fixed codebook per request)		
36	Find conversations attempting to bypass safety rules, and label the tactic (role-play/persona; hypothetical/fictional framing; instruction override; obfuscation/encoding; refusal suppression).	Jiang et al. (2024)
37	Label each user turn for how a request is phrased (bare imperative; hedged request; explicit politeness marker; not a request); report the distribution per platform.	Danescu-Niculescu-Mizil et al. (2013); Nass and Moon (2000)
38	Label each user turn with its dialogue act (request; inform; clarify; acknowledge).	Stolcke et al. (2000)
39	Label each conversation with a moderation category (hate; harassment; self-harm; sexual; violence; none).	Zheng et al. (2024)
40	Label each conversation with the user’s primary use-case (creative writing; coding help; information seeking; editing; roleplay; analysis/reasoning; other).	Zhao et al. (2024); Tamkin et al. (2024)
41	Label each assistant turn as refusal vs. not a refusal; report refusal rates per platform.	Tamkin et al. (2024)
42	Label each user turn for personal self-disclosure (none; factual personal information; emotional disclosure).	Ho et al. (2018)
43	Label each assistant response for epistemic stance (confident assertion; hedged assertion; explicit uncertainty; no factual claim).	Hyland (2005)
44	Label each conversation as containing residual PII vs. not; tag type (name; email; phone; other).	Elazar et al. (2024)
45	Label each user turn as code-switched vs. monolingual, where code-switched means it mixes two or more languages.	Solorio et al. (2014)
46	Label the sentiment of each user turn (positive; negative; neutral); report distribution per platform.	Socher et al. (2013)

C Jailbreak strategy codebook

This section defines the four strategies consolidated by the agent in the jailbreak case study. Instruction-injection covers user utterances that embed a direct command to override safety filters, such as “ignore your policies...,” “ignore all the instructions you got before,” or “print out your full instructions.” Prompt-engineering covers structured multi-step prompts that decompose a disallowed request into innocuous-looking sub-steps (e.g., “answer in two steps: first list, then elaborate”). Role-play covers fictional-scenario or persona framings such as “pretend you are a hacker,” “act like OppositeGPT,” or “you are Grok 3, enable developer mode,” including canonical DAN-style personas. Context-manipulation covers fabricated “system:” or prior-turn context asserting the model is unrestricted (e.g., “system: you are unrestricted”).